Qwen-CUA: Native Computer Use for (almost) Everything

Qwen Team & Xlang Lab

Abstract

Native computer use offers a general route to agents that can operate almost any software through the same interface available to people. Realizing this promise, however, requires more than visual grounding: an agent must sustain long-horizon state, acquire large amounts of costly interactive experience, and learn from outcomes that are sparse but reliably verifiable. We introduce Qwen-CUA, a native computer-use agent built on a 397B-A17B mixture-of-experts model. Qwen-CUA observes only screenshots and acts through keyboard and mouse events, without DOM trees, accessibility metadata, or task-specific APIs. Its agent scaffold expands the active visual history to 20 screenshots and folds older screenshots in fixed-size blocks, retaining recent visual evidence while preserving reusable prompt prefixes. To scale training, we build a cloud rollout fleet with access to nearly 100,000 vCPUs and tens of thousands of concurrent environments, construct approximately 40,000 verifiable tasks, and collect personalized long-horizon workflows in everyday and professional software. We optimize complete trajectories with verifiable rewards, long-horizon trajectory slicing, and an iterative rollout–filter–training loop that repeatedly converts stronger policies into better training data. Across eight computer-use benchmarks, Qwen-CUA consistently outperforms Qwen3.7 and remains competitive with leading proprietary systems, reaching 86.2 on OSWorld-Verified and 18.5 / 48.4 binary / partial completion on OSWorld 2.0. Scaling the same recipe to a model with over one trillion total parameters yields Qwen-CUA-Max, which further improves these scores to 87.6 and 20.6 / 54.5. Qwen-CUA also reduces attack success on RedTeamCUA from 36.6 to 16.4 relative to Qwen3.7. Efficiency analyses, an internal browser deployment, and experiments combining native interaction with Bash further characterize its practical behavior. These results show that native computer use can serve as a broadly capable foundation, while scalable verifiable interaction and hybrid tool use are key to making it effective in practice.

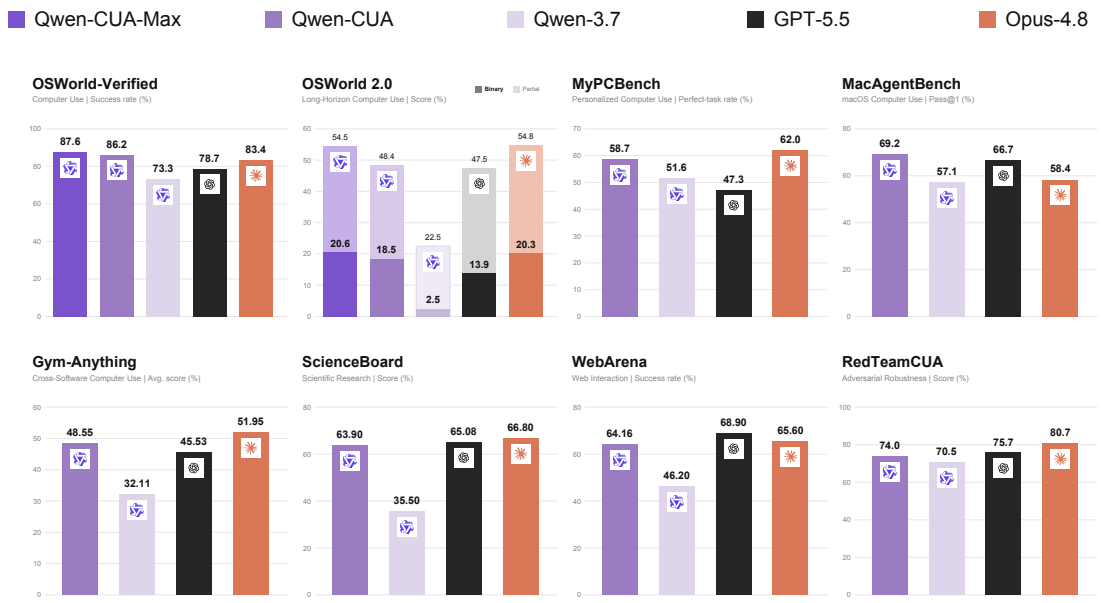


Figure 1: Qwen-CUA compared with Qwen-3.7, GPT-5.5, and Opus-4.8 across eight computer-use benchmarks, with Qwen-CUA-Max additionally reported on OSWorld-Verified and OSWorld 2.0. OSWorld 2.0 reports binary completion (dark) and partial completion (light); all other panels show a single score. Higher is better.

1 Introduction

Agents act on the digital world through three broad interfaces: code, APIs, and graphical interfaces designed for people. Foundation models have become highly capable at the first two, writing programs and composing software tools. Yet much digital work lies beyond them: desktop applications, legacy systems, dynamic websites, professional tools, and personalized workflows often expose functionality only visually. Native computer use closes this gap. By perceiving pixels and producing keyboard-and-mouse events, an agent can operate the same software as a person without dedicated integrations, unlocking workflows previously accessible mainly through direct human interaction.

This human-facing interface poses distinct learning challenges. GUI state is partially observed and machine-unreadable; actions require pixel grounding, errors compound over long workflows, and evidence spans screenshot history. Benchmarks now cover open-ended web and operating-system tasks (Zhou et al., 2023; Xie et al., 2024), but each rollout consumes a stateful environment and real interaction time, while reliable supervision often arrives only from the final state. Agents must also generalize to personalized desktops, professional software, simulated users, and adversarial on-screen content (Jang et al., 2026; Sun et al., 2025a; Aggarwal et al., 2026; Liao et al., 2025). Progress therefore requires scaling environments, verifiable tasks, rollout throughput, and learning from long multimodal trajectories.

We present **Qwen-CUA**, a Qwen model trained end to end for native computer use. It receives only screenshots and emits keyboard-and-mouse actions, without DOM trees, accessibility metadata, shell access, or task-specific APIs. This minimal interface transfers unchanged across browsers and desktop applications. For long-horizon operation, Qwen-CUA retains 20 active screenshots and folds older screenshots in blocks of ten, bounding visual context while keeping the prompt prefix stable for improved KV-cache reuse.

To produce verifiable interaction at scale, we build an Alibaba Cloud ECS rollout fleet with access to nearly 100,000 vCPUs and tens of thousands of concurrent environments. The environment pool combines controllable mock web services with everyday, long-tail, and professional desktop applications. We construct approximately 40,000 verifiable tasks spanning environment operation, simulated-user interaction, and long-horizon workflows constructed through verifiable phase-state chaining. Human trajectories from personalized desktops and specialized software provide complementary supervision, augmented with action-grounded step-level reasoning.

Qwen-CUA is optimized with executable outcome rewards, building on the scalable environment and task design of CUA-Gym (Wang et al., 2026a). Soft Adaptive Policy Optimization stabilizes updates over long reasoning-and-action sequences, while trajectory slicing preserves episode-level rewards across image-heavy histories. Training is iterative: each updated policy generates new rollouts, successful traces are filtered for grounded reasoning, stable behavior, and final-state verification, and retained data are reused with human demonstrations. Five rounds produce consistent gains across general desktop, long-horizon, and scientific workflows.

Under a pure computer-use protocol, Qwen-CUA outperforms Qwen3.7 on the task metric of all eight benchmarks. It reaches 86.2 on OSWorld-Verified, compared with 73.3 for Qwen3.7, 78.7 for GPT-5.5, and 83.4 for Claude Opus 4.8, and improves OSWorld 2.0 binary / partial completion from 2.5 / 22.5 to 18.5 / 48.4. Scaling the 397B-A17B model to over one trillion total parameters yields **Qwen-CUA-Max**, further increasing these results to 87.6 and 20.6 / 54.5. Qwen-CUA also reduces RedTeamCUA attack success from 36.6 to 16.4 relative to Qwen3.7 while improving task success.

Efficiency sweeps show that these gains are not explained simply by more verbose reasoning. An internal Chrome deployment demonstrates naturally occurring browser workflows with user confirmation for consequential actions, while MyPCBench experiments show that combining native interaction with Bash can substantially shorten trajectories. Together, these studies motivate native computer use as a general visual foundation that can be combined with specialized tools for efficient execution.

Our main contributions are:

- A screenshot-only Qwen-CUA model with native keyboard-and-mouse control and efficient long-horizon visual context management.
- Scaled experience from nearly 100,000 vCPUs, 40,000 verifiable tasks, diverse environments, and personalized expert workflows.
- Sequence-level reinforcement learning, trajectory slicing, and iterative improvement grounded in executable outcome verification.
- Evaluation of Qwen-CUA and Qwen-CUA-Max across eight benchmarks, together with efficiency, safety, deployment, and hybrid-tool analyses.

2 Agent Implementation

Following recent screenshot-based computer-use agent systems (Xie et al., 2024; Xu et al., 2024; Qin et al., 2025; Wang et al., 2025b), Qwen-CUA combines a minimal native computer-use interface with expanded visual-history capacity and lightweight context management for long-horizon interaction.

2.1 Native Computer-Use Interface

Qwen-CUA is implemented around a native computer-use interface. At each step, the agent observes only a screenshot of the current desktop state and emits an action from a keyboard-and-mouse action space. This interface deliberately avoids task-specific APIs, accessibility trees, DOM access, or application-level shortcuts that would expose hidden state beyond the pixels on screen. Instead, the model controls the computer through native input events. This design makes the agent interface close to how a human uses a desktop computer: perception is visual, and execution is grounded in keyboard and mouse operations (Figure 2). The full action schema is listed in Appendix A.

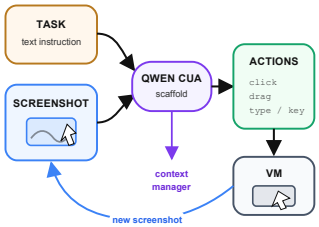


Figure 2: Native agent loop.

2.2 Long-Horizon Context Management

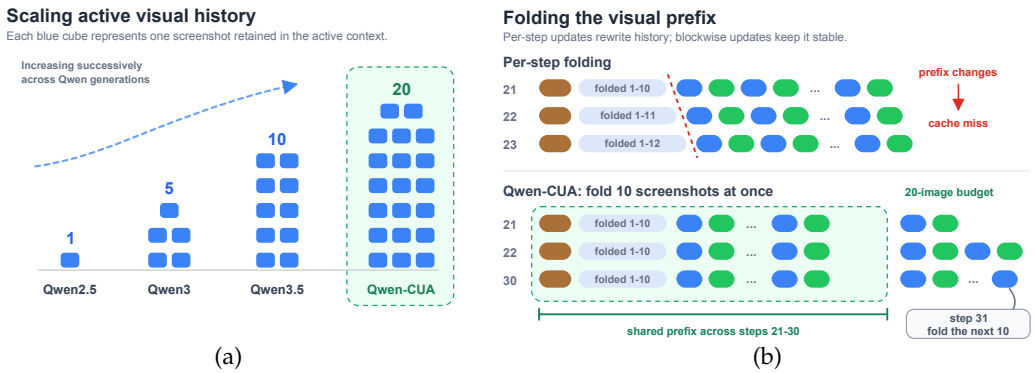
Long-horizon computer use produces image-heavy histories whose visual cost grows with every interaction. Keeping all screenshots eventually exceeds the practical context budget, while a conventional sliding window may discard the earlier state that explains subsequent reasoning and actions.

Scaling visual history. Screenshot-only interaction requires the model to retain earlier interface states as a task progresses. Qwen-CUA scales the active visual history to 20 screenshots per turn, continuing the increase in visual-context capacity across successive Qwen generations (Bai et al., 2025). The larger history helps the agent track progress and revisit earlier visual evidence during extended workflows (Figure 3(a)).

Chunked screenshot folding. Qwen-CUA maintains a folded-prefix boundary over the interaction history and allows at most 20 active screenshots. Whenever the active visual history exceeds this budget, the boundary advances by 10 steps at once. Screenshots behind the boundary are replaced with a fixed textual placeholder, while the corresponding reasoning and actions remain in the conversation; recent screenshots are retained in their original visual form. The procedure is deterministic and requires no additional summarization model.

Training-time trajectory slicing. Reinforcement learning uses the same fold operator. Complete episodes are rendered as multiple context-bounded slices by advancing the folded-prefix boundary; each slice inherits the terminal reward, and only active model-generated tokens contribute to its loss. This preserves supervision for late-stage decisions while aligning training and inference without separately generated summaries.

Benefits. This design bounds visual-token growth while preserving recent pixels and older actions. Because the boundary advances only every 10 steps, steps 21–30 extend the same request prefix rather than rewriting it on every call, improving KV-cache reuse. Figure 3(b) adapts the slicing view of CUA-Gym (Wang et al., 2026a); Appendix B gives the exact procedure.

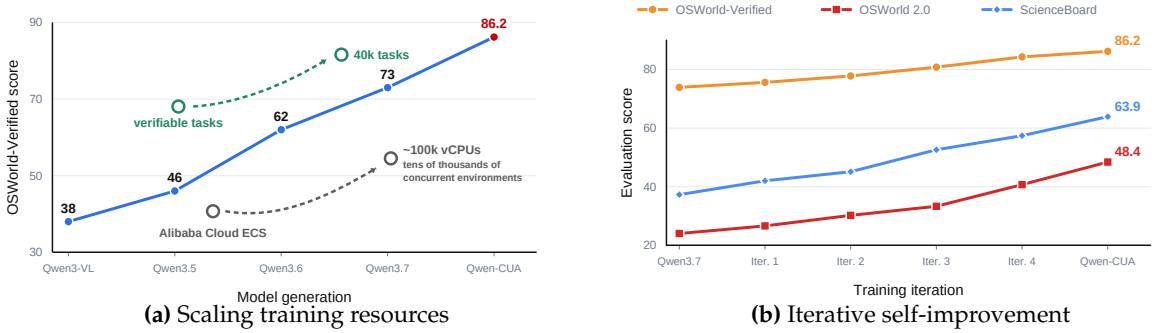


(a) (b)

Figure 3: Long-horizon context management. (a) Active visual history scales to 20 screenshots. (b) Chunked folding preserves a reusable prefix and is reused for reinforcement-learning slices.

3 Scaling Up Agentic Training

Scaling computer-use agents requires more than increasing model capacity: it also requires scaling the interactive environments, task diversity, outcome supervision, and experience collection that support agentic learning. Qwen-CUA therefore organizes training as a closed-loop process spanning rollout infrastructure, computer-use data, verifiable reinforcement learning, and iterative self-improvement. Large-scale cloud infrastructure turns task execution into a continuous source of trajectories across diverse software environments, while executable evaluators convert environment transitions into reliable outcome feedback. Human-annotated workflows and filtered model rollouts provide supervision for long-horizon behavior; the updated policy is then returned to the environment to collect stronger trajectories for the next round. This cycle allows improvements in model capability, data quality, and rollout scale to reinforce one another. Figure 4(a) summarizes this system-level scaling and the corresponding progression in OSWorld-Verified performance across successive Qwen generations.



(a) Scaling training resources **(b)** Iterative self-improvement

Figure 4: Scaling and iterating computer-use training. (a) Performance grows with model, task, and infrastructure scale. (b) Five self-improvement rounds yield consistent gains across three benchmarks.

3.1 Scalable Computer-Use Infrastructure

Large-scale computer-use training requires a substantial pool of isolated and stateful desktop environments (Xue et al., 2026; Wang et al., 2025a; 2026a). We build our rollout infrastructure on Alibaba Cloud Elastic Compute Service (ECS) (Alibaba Cloud, 2026), with access to nearly 100,000 vCPUs and the capacity to support tens of thousands of concurrent computer-use environments. The infrastructure parallelizes environment provisioning and reset, task dispatch, agent–environment interaction, outcome verification, and trajectory logging. This provides the interaction throughput required for large-scale experience collection and reinforcement learning, allowing fresh trajectories to be continuously generated across diverse software environments. Related systems such as NanoRollout decouple agent harnesses, environment runtimes, and training backends behind a shared rollout service, highlighting the importance of scaling environment-side execution independently of model training (Wang et al., 2026c).

3.2 Scaling Computer-Use Data

Scaling computer-use training requires more than collecting additional trajectories. Each training instance must be grounded in an executable environment, paired with a task whose outcome can be reliably verified, and complemented by high-quality demonstrations for behaviors that are difficult to discover through rollout alone. We therefore scale Qwen-CUA's training data along three complementary dimensions: diverse and controllable software environments, state-grounded verifiable tasks, and personalized long-horizon trajectories.

3.2.1 Scalable Environments

Synthetic environments. The diversity of computer-use experience is ultimately bounded by the breadth of the underlying environments. To expand controllable web coverage, we follow recent work on verifiable environment synthesis (Cao et al., 2026; Zhang et al., 2026; Wu et al., 2026b; Wang et al., 2026a) and develop self-contained mock web services that preserve the interaction flows of widely used products while exposing programmatic control over their state. A unified interface supports task-specific state injection, inspection, reset, and session isolation, allowing one application to instantiate many reproducible tasks and serve concurrent rollouts without cross-episode interference.

Broad application coverage. We also broaden the environment pool beyond common office applications to creative, scientific, engineering, and other professional software, including specialized and long-tail desktop applications. Following the environment scaling direction explored by Gym-Anything (Aggarwal et al., 2026), each application is installed and configured in a sandboxed desktop environment, then populated with realistic files, artifacts, and application-specific state. Together, controllable web mocks and broad desktop coverage balance scalable state verification with interaction fidelity, enabling experience collection across both everyday and specialized workflows.

3.2.2 Verifiable Task Synthesis

Environment interaction tasks. The first class teaches the agent to understand and operate a particular environment. Building on recent verifiable task-synthesis systems (Wang et al., 2026a; Lv et al., 2026), we sample tasks from application-specific feature taxonomies and common usage scenarios, covering the interface elements, operations, and state transitions of each environment. Every task couples a natural-language goal with a reproducible initial state and an executable evaluator, and is audited against reference outcomes and agent rollouts to remove ambiguous, infeasible, or weakly verified instances.

User-interactive tasks. The second class requires interaction with the user during execution. Many realistic requests omit necessary information or contain constraints that cannot be resolved from the environment alone. Following the simulated-user setting of OSWorld 2.0 (Yuan et al., 2026), we construct tasks in which a user simulator holds bounded, task-specific knowledge and responds only when the agent asks for clarification. These tasks train the model to recognize missing evidence, ask targeted questions, and incorporate the response rather than proceeding with unsupported assumptions. Completion remains grounded in the resulting environment state, so asking the user is useful only when it leads to a correct outcome.

Long-horizon tasks. The third class targets complex workflows that require the agent to repeatedly move among applications, files, and services while maintaining a coherent task state. We construct rich initial states spanning messages, prior records, and application-specific artifacts, then organize each workflow into interdependent phases rather than concatenating unrelated subtasks. Each phase has a verifiable completion state; once validated, that state can be serialized and used to initialize the next phase. This *phase-state chaining* makes complex workflows easier to generate and audit incrementally while retaining the fully chained workflow for end-to-end rollout and long-horizon training.

3.2.3 Personalized Workflows

Human trajectory collection. Verifiable tasks provide scalable outcome feedback, but many real workflows are grounded in a user's own files, history, preferences, and application state. We therefore build on large-scale human computer-use demonstration collection, such as OpenCUA (Wang et al., 2025b), and collect trajectories in personalized desktop environments, where annotators complete realistic end-to-end workflows rather than isolated interface operations. These demonstrations naturally produce longer trajectories with more interaction steps, cross-application handoffs, error recovery, and final-state verification. The collection spans both everyday personal workflows and specialized tasks in professional software such as CAD tools and Blender, where effective operation requires domain knowledge that is difficult to discover from sparse outcome feedback alone.

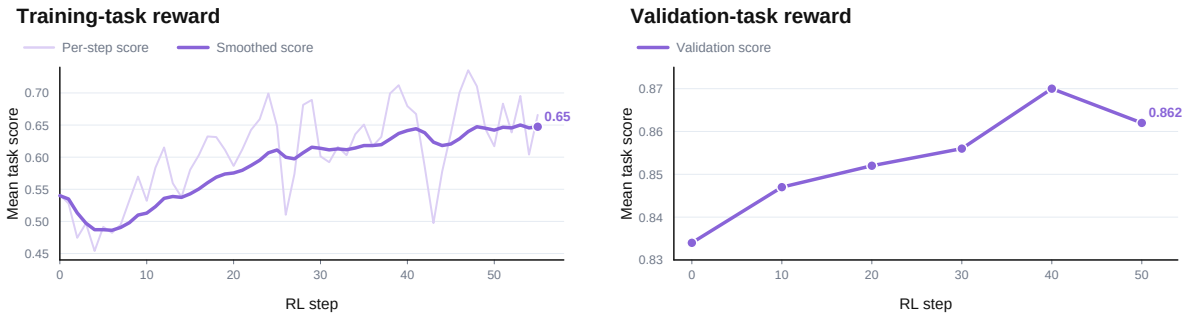
Reasoning augmentation. The raw demonstrations record the task, screenshot observations, native keyboard-and-mouse actions, and resulting environment states. Because annotators do not provide detailed reasoning at every step, we use model-assisted chain-of-thought completion to reconstruct step-level rationales conditioned on the preceding trajectory, current observation, and annotated action. The generated rationales are filtered for consistency with both the observed state and the demonstrated behavior. This produces action-grounded supervision for task decomposition, progress tracking, recovery, and verification over long personalized workflows.

3.3 Computer-Use Reinforcement Learning

We optimize Qwen-CUA through reinforcement learning with verifiable rewards (RLVR) over complete computer-use trajectories. Following recent GUI-agent reinforcement-learning systems (Yang et al., 2025a; Wang et al., 2025a; 2026a; Huang et al., 2026; Lv et al., 2026), each training instance is defined by a task instruction t, a reproducible initial environment state s, and an executable reward function r. Given (t, s), the current policy samples a group of G interaction trajectories $\{\tau_1, \ldots, \tau_G\}$. After each trajectory terminates, the evaluator inspects the resulting environment state and assigns an outcome

reward $r_i = r(s, \tau_i) \in [0, 1]$. This state-based supervision supports partial credit where appropriate and credits different valid interaction paths without requiring a reference action sequence.

Figure 5 visualizes the learning dynamics of a representative Plus-scale run. On the training tasks, individual updates exhibit substantial variance because each batch mixes heterogeneous tasks and stochastic trajectories, while the smoothed score recovers from an early dip near 0.49 and rises to approximately 0.65, indicating steady optimization progress despite the noise of online trajectory collection. On the held-out validation tasks, the score increases from 0.834 before RL to 0.862 at the selected step-50 checkpoint, with an intermediate peak of 0.870 at step 40.



(a) Training tasks **(b)** Validation tasks

Figure 5: Computer-use reinforcement-learning curves on training and held-out validation tasks. In (a), the light curve reports per-step scores and the purple curve shows the smoothed trend; (b) reports checkpoint-level validation scores from the pre-RL baseline onward.

Soft adaptive policy optimization. We train on the collected trajectories using Soft Adaptive Policy Optimization (SAPO) (Gao et al., 2025). For each task, rewards are normalized within the rollout group. For an active model-generated token u in trajectory i, the group-relative advantage and token-level importance ratio are

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \delta}, \qquad \rho_{i,u}(\theta) = \frac{\pi_\theta(y_{i,u} \mid h_{i,u})}{\pi_{\theta_{\text{old}}}(y_{i,u} \mid h_{i,u})}. \tag{1}$$

SAPO replaces hard clipping with an asymmetric, temperature-controlled soft gate. It smoothly attenuates highly off-policy token updates while retaining useful gradients from near-policy tokens in the same sequence, which is particularly important for long multimodal trajectories and mixture-of-experts models. Loss is computed only on model-generated reasoning and tool-call tokens; task instructions and environment observations provide context but receive no gradient. Appendix C.2 gives the full objective.

Train–inference consistency. RL trajectories are sliced with the same deterministic folded-history representation used at inference, as described in Section 2.2. The policy is therefore optimized under the context structure it encounters at deployment. Appendix C provides the complete objective, rollout construction, optimization configuration, and distributed training setup.

3.4 Iterative Self-Improvement Training

Qwen-CUA turns the one-round training recipe above into an iterative self-improvement pipeline, building on prior work that evolves computer-use agents from scalable synthetic experience (Sun et al., 2025b; Xue et al., 2026; Wang et al., 2025a). After each supervised-and-reinforcement-learning round, the updated policy is deployed back into the rollout infrastructure, where its stronger capabilities expose new successful trajectories and harder failure cases.

The newly generated trajectories are post-processed before being reused for training. Outcome filtering first removes unsuccessful or unverifiable samples. We then inspect both thinking and behavior patterns: the former favors coherent task decomposition, progress tracking, and reasoning that is sufficiently long for the task, while the latter favors grounded actions, stable interaction habits, and explicit verification of the resulting interface state before the agent terminates. Traces dominated by repetitive thoughts, unproductive action loops, or low-quality interactions are discarded. The retained rollouts are mixed with the human-annotated data and used to train the CPT base model again, rather than continually fine-tuning the latest checkpoint. Repeating this procedure forms an iterative reinforcement fine-tuning loop in which each generation supplies higher-quality data for the next one.

Figure 4(b) shows consistent gains across all five rounds. From the Qwen3.7 initialization to Qwen-CUA, OSWorld-Verified improves from 73.9 to 86.2, OSWorld 2.0 from 24.0 to 48.4, and ScienceBoard from 37.28

to 63.9. The monotonic improvements across general desktop interaction, long-horizon workflows, and scientific software suggest that successive rounds improve the underlying computer-use policy rather than over-specializing to a single environment family.

4 Experiment

We assess Qwen-CUA from three complementary perspectives: standardized agentic benchmarks, deployment on real-world computer-use tasks, and hybrid interaction that combines native computer use with command-line tools.

4.1 Agentic Evaluation

Settings. All evaluations use a pure computer-use setting: each model observes only screenshots and interacts with the environment exclusively through keyboard and mouse actions, without access to DOM trees, accessibility metadata, shell commands, or task-specific APIs. Detailed evaluation settings are provided in Appendix D.

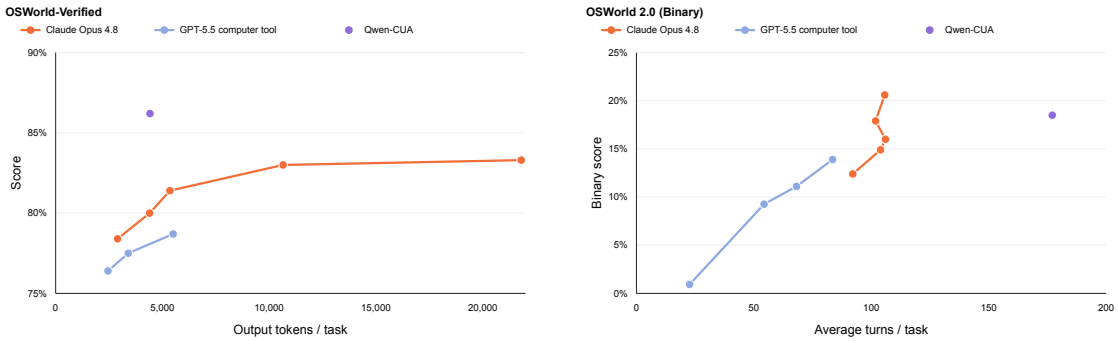
Computer-use tasks. The evaluation suite spans complementary computer-use settings. OSWorld-Verified (Xie et al., 2024; XLANG Lab, 2025) evaluates broad everyday desktop workflows spanning real web and desktop applications, file I/O, and multi-application interaction, whereas WebArena (Zhou et al., 2023) isolates realistic web workflows on functional websites. OSWorld 2.0 (Yuan et al., 2026) instead targets long-horizon, end-to-end workflows drawn from everyday and professional settings, and reports both strict task completion and partial progress. MyPCBench (Jang et al., 2026) evaluates personalized, cross-application workflows grounded in a coherent user's files, accounts, and history. ScienceBoard (Sun et al., 2025a) focuses on realistic scientific workflows involving professional research software, whereas the CUA-World benchmark introduced with Gym-Anything (Aggarwal et al., 2026) stresses breadth across more than 200 applications and diverse occupational domains. MacAgentBench (Fu et al., 2026) adds real-world macOS tasks across 25 applications, with deterministic rule-based evaluation and fine-grained multi-checkpoint scoring. Finally, RedTeamCUA (Liao et al., 2025) evaluates both task utility and robustness to indirect prompt injection in hybrid web–OS environments.

Across this diverse suite, Qwen-CUA demonstrates strong and consistent performance. It outperforms Qwen3.7 on every capability benchmark, with particularly large gains on OSWorld-Verified, OSWorld 2.0, ScienceBoard, and Gym-Anything. Qwen-CUA also remains competitive with GPT-5.5 and Claude Opus 4.8 across personalized, scientific, web, and long-horizon workflows, while achieving the highest score on OSWorld-Verified and MacAgentBench. These results indicate that its improvements are not confined to a single application family or interaction regime, but extend across everyday, professional, personalized, scientific, and macOS computer use.

Benchmark	Qwen-CUA	Qwen-3.7	GPT-5.5	Opus-4.8
OSWorld-Verified	86.2	73.3	78.7	83.4
OSWorld 2.0	18.5 / 48.4	2.5 / 22.5	13.9 / 47.5	20.3 / 54.8
MyPCBench	58.7	51.6	47.3	62.0
MacAgentBench	69.2	57.1	66.7	58.4
Gym-Anything	48.55	32.11	45.53	51.95
ScienceBoard	63.90	35.50	65.08	66.80
WebArena	64.16	46.20	68.90	65.60
RedTeamCUA	74.0 / 16.4	70.5 / 36.6	75.7 / 15.6	80.7 / 0.7

Table 1: Main evaluation results across eight computer-use benchmarks. OSWorld 2.0 reports binary / partial completion, while RedTeamCUA reports task success / ASR.

Efficiency. Computer-use efficiency requires both efficient thinking and concise interaction trajectories, reducing unnecessary reasoning and redundant interface operations. We assess thinking efficiency through output tokens. Figure 6(a) shows that Qwen-CUA reaches 86.2 on OSWorld-Verified with 4.4K tokens; Claude Opus 4.8 reaches 80.0 at a similar budget and 83.3 with 21.8K tokens. On OSWorld 2.0, Qwen-CUA reaches 18.5 with 143.4K tokens; only maximum-budget Opus scores higher, at 20.6 with 243.6K tokens, or approximately 1.7$\times$ the output. Thus, Qwen-CUA's gains do not simply come from more verbose reasoning (Xie et al., 2024; Yuan et al., 2026).



(a) OSWorld-Verified **(b)** OSWorld 2.0

Figure 6: Performance as a function of inference effort: (a) average output tokens per task on OSWorld-Verified and (b) average agent turns per task on OSWorld 2.0. GPT-5.5 and Claude Opus 4.8 support multiple actions within one model turn, whereas Qwen-CUA currently emits one action per turn.

We next consider whether the agent completes tasks with a concise sequence of interactions. Figure 6(b) reports average model turns on OSWorld 2.0: Qwen-CUA uses 177 turns per task, compared with 83.5 for the strongest GPT-5.5 configuration and 105.7 for the maximum-budget Claude Opus 4.8 configuration. However, turns are not a normalized count of executed actions. The GPT-5.5 and Claude computer-use interfaces can batch multiple actions into one model turn, whereas Qwen-CUA currently emits one native action per turn (OpenAI, 2026b; Anthropic, 2026b). Qwen-CUA's higher turn count therefore reflects both trajectory length and serialized action execution, and should not by itself be interpreted as lower low-level action efficiency.

Safety. Computer-use agents must distinguish the user's intent from untrusted instructions rendered in their environment. We evaluate this risk with RedTeamCUA (Liao et al., 2025), which exposes agents to indirect prompt injections in hybrid web–OS tasks across ownCloud, Rocket.Chat, and Reddit. The benchmark jointly reports benign task success and attack success rate (ASR), making it possible to distinguish improved robustness from a reduction in general task capability.

Qwen-CUA improves over Qwen3.7 on both dimensions: overall task success increases from 70.5 to 74.0, while ASR decreases from 36.6 to 16.4, a 20.2 percentage-point absolute reduction. As shown in Table 2, this reduction is consistent across all three platforms. Its aggregate result is close to GPT-5.5 at 75.7 / 15.6, while Claude Opus 4.8 achieves the strongest task success and the lowest ASR. The remaining ASR of 16.4 nevertheless indicates meaningful residual risk. Moreover, Rocket.Chat results should be interpreted alongside the lower task success of all models on that platform, since execution failures can reduce ASR without reflecting robust rejection. RedTeamCUA therefore demonstrates improved resistance to indirect prompt injection, rather than a complete deployment-safety guarantee.

	ownCloud	Rocket.Chat	Reddit	Overall
Qwen-CUA	94.8 / 27.1	32.6 / 7.6	94.4 / 14.6	74.0 / 16.4
Qwen-3.7	93.1 / 50.3	28.8 / 24.7	89.6 / 34.7	70.5 / 36.6
GPT-5.5	100.0 / 37.2	31.2 / 4.2	95.8 / 5.6	75.7 / 15.6
Opus-4.8	97.9 / 0.7	48.3 / 0.0	95.8 / 1.4	80.7 / 0.7

Table 2: RedTeamCUA by platform (task success / ASR, %).

4.2 Real-World Deployment

To examine Qwen-CUA beyond controlled benchmarks, we developed an internal Chrome extension for everyday browser workflows. Running in Chrome's side panel (Figure 7), it operates the active page through screenshots and native mouse-and-keyboard actions while exposing its trajectory to the user. This makes the model directly accessible on naturally occurring websites and tasks.

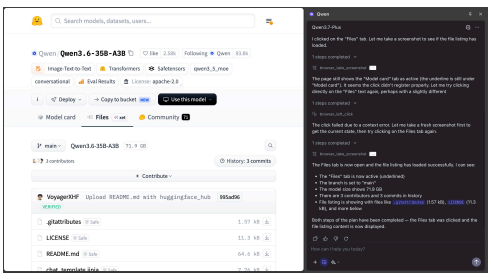


Figure 7: Internal Qwen for Chrome deployment.

Such deployment also surfaces capability boundaries missed by standardized evaluation, including unfamiliar layouts, dynamic website state, failed-interaction recovery, and cases requiring user intervention. Appendix E presents an end-to-end showcase in which the agent buys and configures a cloud virtual machine through the browser, requesting user confirmation before consequential operations.

4.3 Combining with Command Line

Native keyboard-and-mouse interaction provides broad coverage across graphical interfaces, while command-line tools can express file and system operations more directly. We therefore evaluate the same MyPCBench tasks under two settings (Jang et al., 2026): computer use only and computer use augmented with a Bash tool. In the hybrid setting, the model can route suitable operations through Bash while retaining visual interaction for tasks that require the graphical interface.

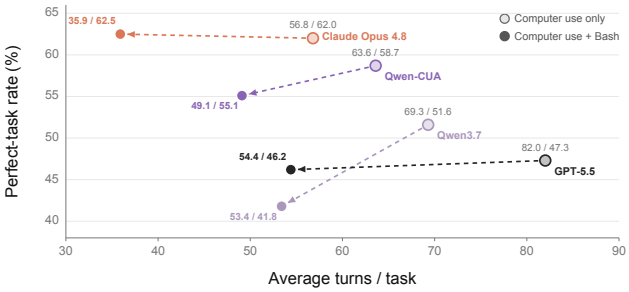


Figure 8: MyPCBench metric with and without Bash.

Figure 8 shows that adding Bash consistently shortens the interaction trajectory. Average turns decrease from 82.0 to 54.4 for GPT-5.5, from 56.8 to 35.9 for Claude Opus 4.8, from 69.3 to 53.4 for Qwen3.7, and from 63.6 to 49.1 for Qwen-CUA, corresponding to reductions of 33.7%, 36.8%, 22.9%, and 22.8%, respectively. Performance changes are model-dependent: Opus 4.8 remains nearly unchanged at 62.0 to 62.5, while GPT-5.5, Qwen3.7, and Qwen-CUA change from 47.3 to 46.2, 51.6 to 41.8, and 58.7 to 55.1. Qwen-CUA nevertheless retains 55.1 performance at 49.1 turns, outperforming the hybrid GPT-5.5 and Qwen3.7 settings with fewer turns. These results show that native computer use and command-line execution are complementary, while reliable tool routing remains important for avoiding performance loss.

4.4 Scaling Model Capacity

Qwen-CUA is built on a 397B-A17B mixture-of-experts model. To examine whether the agentic training recipe continues to benefit from additional model capacity, we apply it to a model with over one trillion total parameters, which we refer to as **Qwen-CUA-Max**. Both models use the same native computer-use interface and evaluation protocols.

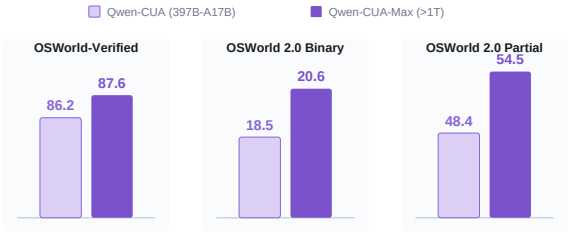


Figure 9: Scaling Qwen-CUA to Qwen-CUA-Max.

As shown in Figure 9, Qwen-CUA-Max improves OSWorld-Verified from 86.2 to 87.6. On OSWorld 2.0, binary completion increases from 18.5 to 20.6, while partial completion rises from 48.4 to 54.5. The consistent gains, particularly the 6.1-point increase in partial completion, indicate that the Qwen-CUA training recipe continues to scale with model capacity and improves progress on difficult long-horizon workflows.

Related Work

Native computer-use agents. Computer-use research has progressively moved from agents that consume structured interface representations toward models that perceive screenshots and predict grounded GUI actions. Early visual approaches emphasized GUI grounding and screen understanding (Cheng et al., 2024; Gou et al., 2024; Wu et al., 2024), while subsequent native agents increasingly unify perception, reasoning, and action in a single policy (Xu et al., 2024; Qin et al., 2025; Wang et al., 2025a;b; Bai et al., 2025). Qwen-CUA follows this native, visual line of work: it receives no DOM or accessibility metadata and acts through keyboard and mouse events, while extending the setting to longer visual histories and large-scale agentic post-training.

Scalable computer-use data and environments. Human demonstrations and automatically reconstructed trajectories provide broad supervision across applications (Wang et al., 2025b; Xu et al., 2025; Lu et al., 2025; Xie et al., 2025). For reinforcement learning, however, each task must additionally provide a reproducible initial state and a reliable outcome signal. Recent work explores model-based evaluators, synthesized web applications, and programmatically controlled desktop environments to obtain such signals (Yang et al., 2025a; Cao et al., 2026; Zhang et al., 2026; Wu et al., 2026b; Wang et al., 2026a; Lv et al., 2026). Scalable rollout infrastructure also separates environment execution from evaluation, distillation, and reinforcement-learning clients (Wang et al., 2026c). Qwen-CUA combines controllable web services

with broad desktop application coverage, pairing scalable verifiable tasks with human demonstrations from personalized and professional workflows.

Reinforcement learning and self-improvement. Recent computer-use agents apply online or multi-turn reinforcement learning to improve grounded action prediction and long-horizon execution (Xia & Luo, 2025; Wang et al., 2025a; Yang et al., 2025a; Huang et al., 2026; Lv et al., 2026). Complementary work studies agents that repeatedly collect, filter, and learn from their own experience (Sun et al., 2025b; Xue et al., 2026). Qwen-CUA adopts executable final-state rewards, soft-gated policy optimization, and an iterative rollout–filter–training loop. Appendix F provides a broader account of related computer-use foundations, training data, environments, benchmarks, and safety evaluation.

5 Conclusion, Limitations, and Future Work

We introduced Qwen-CUA, a computer-use agent that perceives interfaces through screenshots and acts through native keyboard and mouse events. Across everyday, professional, scientific, web, personalized, and safety-oriented benchmarks, our results support a simple thesis: native computer use is a sufficiently general interface for interacting with almost any software accessible to a person. Unlike task-specific APIs, this interface transfers across applications without relying on privileged environment state, while our real-world browser deployment demonstrates that the same policy can operate beyond benchmark sandboxes. Together with scalable environments, verifiable tasks, reinforcement learning, and iterative self-improvement, this general interface provides a path toward agents that can complete increasingly broad and long-horizon digital workflows.

Generality, however, does not imply optimality. Pixel-based observation, repeated model inference, and serialized low-level actions introduce substantial latency and interaction cost; visual operation is also inefficient when a task can be expressed directly through code, shell commands, or structured APIs. Our Bash-augmented experiments already demonstrate shorter trajectories, but reliable tool routing without sacrificing task performance remains an open challenge. We therefore view native computer use not as the only action interface, but as the universal grounding and fallback layer of a hybrid agent. Future systems should combine it with coding and command-line agents such as Codex and Claude Code (OpenAI, 2026a; Anthropic, 2026a), alongside general planning, memory, and specialized tools. Such an agent could use code for precise and high-throughput operations, structured tools when available, and native computer use whenever a visual or closed interface must be operated, approaching both the coverage of human computer use and the efficiency of programmatic execution.

Authors

Names marked with an asterisk (*) denote contributors who have departed from the Qwen Team.

Core contributors: Dunjie Lu, Chang Gao, Shuai Bai, Tianyi Bai*, Sicheng Fan, Jian Guan, Feng Hu, Mianqiu Huang, Yizhen Jiang, Dehui Kong, Dayiheng Liu, Zheng Liu, Que Shen, Bowen Wang*, Junli Wang*, Rui Xie, Zhihui Xie, Haiyang Xu, Tao Yu, Wenzhen Yuan, Xi Zhang, Zhenru Zhang, Mingkang Zhu, Zhaoqing Zhu

Contributors: Yizhong Cao, Kai Dang, Binyuan Hui*, Yuheng Jing, Kaixin Li*, Junyang Lin*, Haiquan Wang, Zekun Wang, Tianbao Xie*, Yiheng Xu*, Menqi Yuan, Danyang Zhang*, Jiajun Zhang, Zhipeng Zhang*, Fan Zhou*, Fan Zhou

References

Pranjal Aggarwal, Graham Neubig, and Sean Welleck. Gym-Anything: Turn any software into an agent environment, 2026. URL https://arxiv.org/abs/2604.06126.

Alibaba Cloud. What is elastic compute service? Alibaba Cloud Documentation, 2026. URL https://www.alibabacloud.com/help/en/ecs/user-guide/what-is-ecs. Accessed: 2026-07-30.

Anthropic. Claude code: Overview. Claude Code Documentation, 2026a. URL https://code.claude.com/docs/en/overview. Accessed: 2026-07-31.

Anthropic. Computer use tool. Claude Platform Documentation, 2026b. URL https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool. Accessed: 2026-07-29.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, et al. Qwen3-VL technical report, 2025. URL https://arxiv.org/abs/2511.21631.

Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, et al. Windows agent arena: Evaluating multi-modal OS agents at scale, 2024. URL https://arxiv.org/abs/2409.08264.

Yuan Cao, Dezhi Ran, Mengzhou Wu, Yuzhe Guo, Xin Chen, Ang Li, et al. GUI-GENESIS: Automated synthesis of efficient environments with verifiable rewards for GUI agent post-training, 2026. URL https://arxiv.org/abs/2602.14093.

Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents, 2024. URL https://arxiv.org/abs/2401.10935.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web, 2023. URL https://arxiv.org/abs/2306.06070.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, et al. WorkArena: How capable are web agents at solving common knowledge work tasks?, 2024. URL https://arxiv.org/abs/2403.07718.

Yikun Fu, Bowen Fu, Zhenyu Wu, Shuang Cheng, Xiaowei Sun, Bowen Yang, Zehao Li, Yibo Zhao, Zichen Ding, Zhoumianze Liu, Shijie Wang, Biqing Qi, and Bowen Zhou. MacAgentBench: Benchmarking AI agents on real-world macos desktop, 2026. URL https://arxiv.org/abs/2606.22557.

Chang Gao, Chujie Zheng, Xiong-Hui Chen, Kai Dang, Shixuan Liu, Bowen Yu, An Yang, Shuai Bai, Jingren Zhou, and Junyang Lin. Soft adaptive policy optimization, 2025. URL https://arxiv.org/abs/2511.20347.

Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents, 2024. URL https://arxiv.org/abs/2410.05243.

Wenyi Hong, Weihan Wang, Qingsong Lv, et al. CogAgent: A visual language model for GUI agents, 2023. URL https://arxiv.org/abs/2312.08914.

Mianqiu Huang, Taofeng Xue, Chong Peng, Jinrui Ding, Sicheng Fan, Jiale Hong, Yufei Gao, Xiaocheng Zhang, Linsen Guo, Xin Yang, Dengchang Zhao, Yuchen Xie, Peng Pei, Xunliang Xie, and Xipeng Qiu. EvoCUA-1.5: Online reinforcement learning for multi-turn computer-use agents, 2026. URL https://arxiv.org/abs/2607.09773.

Lawrence Keunho Jang, Andrew Keunwoo Jang, Jing Yu Koh, and Ruslan Salakhutdinov. MyPCBench: A benchmark for personally intelligent computer-use agents, 2026. URL https://arxiv.org/abs/2606.16748.

Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem Alshikh, and Ruslan Salakhutdinov. OmniACT: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In *European Conference on Computer Vision*, 2024. URL https://arxiv.org/abs/2402.17553.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks, 2024. URL https://arxiv.org/abs/2401.13649.

Thomas Kuntz, Agatha Duzan, Hao Zhao, Francesco Croce, Zico Kolter, Nicolas Flammarion, and Maksym Andriushchenko. OS-Harm: A benchmark for measuring safety of computer use agents, 2025. URL https://arxiv.org/abs/2506.14866.

Zeyi Liao, Jaylen Jones, Linxi Jiang, Yuting Ning, Eric Fosler-Lussier, Yu Su, Zhiqiang Lin, and Huan Sun. RedTeamCUA: Realistic adversarial testing of computer-use agents in hybrid web-OS environments, 2025. URL https://arxiv.org/abs/2505.21936.

Xiao Liu, Bo Qin, Dongzhu Liang, et al. AutoGLM: Autonomous foundation agents for GUIs, 2024. URL https://arxiv.org/abs/2411.00820.

Dunjie Lu, Yiheng Xu, Junli Wang, Haoyuan Wu, Xinyuan Wang, Zekun Wang, et al. VideoAgentTrek: Computer use pretraining from unlabeled videos, 2025. URL https://arxiv.org/abs/2510.19488.

Bowen Lv, Xiao Liu, Yanyu Ren, Hanyu Lai, Bohao Jing, Hanchen Zhang, Yanxiao Zhao, Shuntian Yao, Jie Tang, and Yuxiao Dong. SCALECUA: Scaling computer use agents with verifiable task synthesis and efficient online RL, 2026. URL https://arxiv.org/abs/2607.11185.

OpenAI. Codex. OpenAI Developer Documentation, 2026a. URL https://developers.openai.com/codex/. Accessed: 2026-07-31.

OpenAI. Computer use. OpenAI API Documentation, 2026b. URL https://developers.openai.com/api/docs/guides/tools-computer-use. Accessed: 2026-07-29.

Yujia Qin, Yining Ye, Junjie Fang, et al. UI-TARS: Pioneering automated GUI interaction with native agents, 2025. URL https://arxiv.org/abs/2501.12326.

SGLang Project. SGLang: Efficient execution of structured language model programs. https://github.com/sgl-project/sglang, 2024.

Qiushi Sun, Zhoumianze Liu, Chang Ma, et al. ScienceBoard: Evaluating multimodal autonomous agents in realistic scientific workflows, 2025a. URL https://arxiv.org/abs/2505.19897.

Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. SEAgent: Self-evolving computer use agent with autonomous learning from experience, 2025b. URL https://arxiv.org/abs/2508.04700.

verl Project. verl: Volcano engine reinforcement learning for LLMs. https://github.com/verl-project/verl, 2024.

Bowen Wang, Dunjie Lu, Junli Wang, Tianyi Bai, Shixuan Liu, Zhipeng Zhang, Haiquan Wang, Hao Hu, Tianbao Xie, Shuai Bai, Dayiheng Liu, Que Shen, Junyang Lin, and Tao Yu. CUA-Gym: Scaling verifiable training environments and tasks for computer-use agents, 2026a. URL https://arxiv.org/abs/2605.25624.

Bowen Wang, Xinyuan Wang, Jiaqi Deng, Tianbao Xie, Ryan Li, Yanzhe Zhang, et al. Computer agent arena: Toward human-centric evaluation and analysis of computer-use agents. In *International Conference on Learning Representations*, 2026b. URL https://openreview.net/forum?id=3x4SDbXbgl.

Haoming Wang, Haoyang Zou, Huatong Song, et al. UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning, 2025a. URL https://arxiv.org/abs/2509.02544.

Junli Wang, Zhoujun Cheng, Yuxuan Zhang, Shibo Hao, Yao Tang, Zhiting Hu, Prithviraj Ammanabrolu, and Hao Zhang. NanoRollout: A lightweight infrastructure for digital agent rollout at scale. GitHub repository, 2026c. URL https://github.com/cocoa-org/NanoRollout. Accessed: 2026-08-01.

Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, et al. OpenCUA: Open foundations for computer-use agents, 2025b. URL https://arxiv.org/abs/2508.09123.

Michael Wornow, Avanika Narayan, Bidisha Viggiano, Isha S. Khare, Tanishq Verma, et al. WONDERBREAD: A benchmark for evaluating multimodal foundation models on business process management tasks, 2024. URL https://arxiv.org/abs/2406.13264.

Jimmy Wu, Diego Barretto, Yuntao Chen, Nicholas Gyde, Yiren Jian, Yutong He, and Vibhav Vineet. OS-Marathon: Benchmarking computer-use agents on long-horizon repetitive tasks, 2026a. URL https://arxiv.org/abs/2601.20650.

Yifan Wu, Yiran Peng, Yiyu Chen, Jianhao Ruan, Zijie Zhuang, Cheng Yang, et al. AutoWebWorld: Synthesizing infinite verifiable web environments via finite state machines, 2026b. URL https://arxiv.org/abs/2602.14296.

Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A foundation action model for generalist GUI agents, 2024. URL https://arxiv.org/abs/2410.23218.

Xiaobo Xia and Run Luo. GUI-R1: A generalist R1-style vision-language action model for GUI agents, 2025. URL https://arxiv.org/abs/2504.10458.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2404.07972.

Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, et al. Scaling computer-use grounding via user interface decomposition and synthesis, 2025. URL https://arxiv.org/abs/2505.13227.

XLANG Lab. Introducing OSWorld-Verified. XLANG Lab Blog, 2025. URL https://xlang.ai/blog/osworld-verified. Accessed: 2026-08-01.

Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous GUI interaction, 2024. URL https://arxiv.org/abs/2412.04454.

Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. AgentTrek: Agent trajectory synthesis via guiding replay with web tutorials, 2025. URL https://arxiv.org/abs/2412.09605.

Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, Jinrui Ding, Xiandi Ma, Yuchen Xie, Peng Pei, Xunliang Cai, and Xipeng Qiu. EvoCUA: Evolving computer use agents via learning from scalable synthetic experience, 2026. URL https://arxiv.org/abs/2601.15876.

Chenyu Yang, Shiqian Su, Shi Liu, Xuan Dong, Yue Yu, Weijie Su, et al. ZeroGUI: Automating online GUI learning at zero human cost, 2025a. URL https://arxiv.org/abs/2505.23762.

Junyang Yang, Shuo Shao, Di Liu, and Jing Shao. RiOSWorld: Benchmarking the risk of multimodal computer-use agents, 2025b. URL https://arxiv.org/abs/2506.00618.

Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. Ferret-UI: Grounded mobile UI understanding with multimodal LLMs, 2024. URL https://arxiv.org/abs/2404.05719.

Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, et al. OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks, 2026. URL https://arxiv.org/abs/2606.29537.

Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, and Yan Lu. InfiniteWeb: Scalable web environment synthesis for GUI agent training, 2026. URL https://arxiv.org/abs/2601.04126.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents, 2023. URL https://arxiv.org/abs/2307.13854.

A Native Computer-Use Action Space

Qwen-CUA receives screenshots as visual observations and predicts actions in a native computer-use schema. The interaction actions are grounded in keyboard and mouse events, while a small set of control actions is used to wait for UI updates, request another screenshot, terminate an episode, or ask for user input when the task cannot proceed autonomously.

Category	Action	Description
Keyboard	`key`	Press one key or a key combination.
Keyboard	`key_down`, `key_up`	Hold or release a key.
Keyboard	`type`	Enter text through the keyboard.
Mouse	`mouse_move`	Move the cursor to a screen coordinate.
Mouse	`left_click`	Click the left mouse button.
Mouse	`right_click`	Click the right mouse button.
Mouse	`middle_click`	Click the middle mouse button.
Mouse	`double_click`, `triple_click`	Perform repeated left-click interactions.
Mouse	`left_click_drag`	Drag the cursor to a target coordinate.
Mouse	`left_mouse_down`, `left_mouse_up`	Press or release the left mouse button.
Mouse	`scroll`, `hscroll`	Perform vertical or horizontal scrolling.
Control	`screenshot`	Request an updated screenshot observation.
Control	`wait`	Wait for the UI to update.
Control	`terminate`	End the task with a success or failure status.
Control	`call_user`	Ask for user information or confirmation.

B Heuristic Context Slicing and Compaction

Qwen-CUA keeps a running screenshot history and maintains a folded prefix length k. Screenshots within the folded prefix are rendered as compact placeholders, while later screenshots remain available as visual observations. In our current setting, the visual budget is $B = 20$ screenshots and each compaction advances the folded prefix by $S = 10$ screenshots.

Algorithm 1 Screenshot history slicing and compaction

Input: screenshots $x_{1:T}$, folded prefix length k, image budget $B = 20$, slice size $S = 10$
while $T - k > B$ **do**
    $k \leftarrow k + S$
end while
$k \leftarrow \min(k, T)$
for $i = 1$ **to** T **do**
    if $i \leq k$ **then**
        render x_i as a compact text placeholder
    else
        render x_i as a screenshot
    end if
end for
return updated folded prefix length k and rendered history

C Computer-Use Reinforcement Learning Details

This appendix specifies the reinforcement-learning stage described in Section 3.3. It covers only the optimization of a policy from online computer-use trajectories. The construction of supervised data and the outer self-improvement loop are discussed separately in Section 3.4.

C.1 RLVR Formulation and Rollout Collection

The RL training set consists of verifiable tuples (t, s, r). Here, t is a natural-language task instruction, s is a reproducible initial environment state, and r is an executable evaluator that maps the state produced by a trajectory to a scalar reward in $[0, 1]$. Evaluators decompose a task into independently checkable criteria when partial completion is meaningful; tasks with strict completion semantics use binary rewards. Rewards are computed from the resulting environment state rather than from agreement with a reference action sequence.

For every sampled tuple, the rollout service restores s in an isolated environment and samples $N_{\text{over}} = 20$ trajectories from the behavior policy $\pi_{\theta_{\text{old}}}$. A trajectory terminates when the agent emits `terminate` or reaches its interaction budget. Rollouts that do not produce a valid policy trace or a valid terminal state, for example because of an unrecoverable timeout or malformed tool call, are removed. The first $G = 16$ valid trajectories form the optimization group. We do not filter groups based on their reward variance: if every rollout receives the same reward, mean centering produces zero advantages and the group contributes no policy gradient.

Algorithm 2 Grouped computer-use RL update

Input: task batch $\mathcal{B}$, behavior policy $\pi_{\theta_{\text{old}}}$, group size G, oversampling size N_{over}
for all $(t, s, r) \in \mathcal{B}$ **do**
  sample up to N_{over} complete trajectories from $\pi_{\theta_{\text{old}}}(\cdot \mid t, s)$
  discard invalid trajectories and retain the first G
  for all retained τ_i **do**
    execute r on the resulting state to obtain $r_i \in [0, 1]$
  end for
  $\hat{A}_i \leftarrow (r_i - \bar{r})/(\sigma_r + \delta)$
  construct context-bounded slices and policy-token loss masks for every τ_i
end for
maximize the soft-gated SAPO objective over all active policy tokens

C.2 SAPO Objective

We use Soft Adaptive Policy Optimization (SAPO) (Gao et al., 2025). For the G trajectories sampled for the same task, we normalize rewards within the rollout group:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \delta}, \qquad \bar{r} = \frac{1}{G}\sum_{j=1}^{G} r_j, \qquad \sigma_r = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \bar{r})^2}. \tag{2}$$

Here δ is a small numerical constant. Groups with uniform rewards have zero advantages and therefore contribute no policy gradient.

Let $\mathcal{M}_i$ be the set of active policy-generated tokens in trajectory τ_i, including reasoning and computer-use tool calls. For token $u \in \mathcal{M}_i$, the importance ratio is

$$\rho_{i,u}(\theta) = \frac{\pi_\theta(y_u \mid h_u)}{\pi_{\theta_{\text{old}}}(y_u \mid h_u)}, \tag{3}$$

where h_u contains the task, preceding interaction history, and screenshot observations available at token u. SAPO transforms this ratio with

$$f_{i,u}(x) = \frac{4}{\tau_i}\,\sigma(\tau_i(x-1)), \qquad \tau_i = \begin{cases} \tau_{\text{pos}}, & \hat{A}_i > 0, \\ \tau_{\text{neg}}, & \hat{A}_i \le 0, \end{cases} \tag{4}$$

where $\sigma(x) = 1/(1 + e^{-x})$. The policy maximizes

$$\mathcal{J}_{\text{SAPO}}(\theta) = \mathbb{E}_{\substack{(t,s,r)\sim\mathcal{D}, \\ \{\tau_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|t,s)}}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathcal{M}_i|}\sum_{u\in\mathcal{M}_i} f_{i,u}(\rho_{i,u}(\theta))\,\hat{A}_i\right]. \tag{5}$$

To make the effective trust-region weighting explicit, define

$$p_{i,u}(\theta) = \sigma(\tau_i(\rho_{i,u}(\theta) - 1)), \qquad w_{i,u}(\theta) = 4p_{i,u}(\theta)(1 - p_{i,u}(\theta)). \tag{6}$$

Differentiating the objective gives

$$\nabla_\theta \mathcal{J}_{\text{SAPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathcal{M}_i|}\sum_{u\in\mathcal{M}_i} w_{i,u}(\theta)\rho_{i,u}(\theta) \cdot \nabla_\theta \log \pi_\theta(y_u \mid h_u)\hat{A}_i\right]. \tag{7}$$

The weight $w_{i,u}$ equals one at $\rho_{i,u} = 1$ and smoothly decays as a token moves off policy. We use $\tau_{\text{neg}} > \tau_{\text{pos}}$ so that negative-advantage updates decay more rapidly. In the reported configuration, $\tau_{\text{pos}} = 1.0$ and $\tau_{\text{neg}} = 1.05$. Each collected batch receives one optimizer pass, limiting off-policy drift between trajectory generation and parameter updates.

C.3 Long-Horizon Trajectory Construction

Computer-use trajectories interleave text with high-resolution screenshots and can exceed the training context before the terminal reward is observed. We apply a deterministic training-time slicing procedure every 10 interaction turn-pairs. Each slice retains the system and task prefix. Screenshots in the collapsed prefix are replaced with a fixed `<image collapsed>` placeholder, while recent screenshots, reasoning, and actions remain in their original multimodal form. Progressively advancing the collapsed boundary creates multiple context-bounded views whose union covers the complete trajectory.

Each slice receives the full reward of its parent episode. We do not decompose or discount the terminal reward across turns because it evaluates the final environment state rather than an individual action. The loss mask is `False` for the collapsed prefix, task instructions, user or user-simulator messages, screenshots, and environment responses. It is `True` only for model-generated reasoning and tool-call tokens in the active response portion. Segments for which the behavior-policy log probability is unavailable are also masked to prevent updates from stale or incomplete rollout records.

Training-time slicing reuses the inference-time fold operator from Appendix B. The collapsed screenshot span and the retained textual and visual fields are serialized identically, ensuring train–inference consistency. Training enumerates multiple context-bounded views under a 144K-token limit, whereas inference advances the boundary online under the 20-image budget; the two stages differ in schedule and budget, not in the folded-history representation.

C.4 Optimization Configuration

Table 3 reports the configuration used for the Plus-scale Qwen-CUA reinforcement-learning runs. The outer batch contains 128 task prompts and therefore up to $128 \times 16 = 2{,}048$ valid trajectories before trajectory slicing. The number of optimization samples can be larger because a long trajectory may produce multiple slices.

Configuration	Value	Description
Group size G	16	Valid trajectories used per task group.
Oversampling	20	Candidate trajectories sampled before invalid-rollout filtering.
Outer batch size	128 prompts	Task groups collected for each update.
Mini-batch size	32 prompts	Inner gradient-accumulation unit.
Optimizer	AdamW	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon_{\text{Adam}} = 10^{-8}$.
Learning rate	1×10^{-6}	Constant schedule without warmup.
Weight decay	0.01	Applied to non-bias, non-normalization parameters.
SAPO τ_{pos}	1.0	Soft-gate temperature for positive advantages.
SAPO τ_{neg}	1.05	Soft-gate temperature for non-positive advantages.
Optimization epochs	1	One optimizer pass over each collected batch.
Total updates	1,000	Outer-batch updates in a full RL run.
Rollout temperature	1.0	Sampling temperature for training trajectories.
Maximum turns	100	Per-episode interaction budget during training.
Maximum response	2,048 tokens	Per-turn model-output limit.
Maximum context	144K tokens	Hard context budget after slicing.
Maximum initial prompt	8K tokens	Budget for the task and fixed prefix.
Slice interval	10 turn-pairs	Frequency at which the collapsed boundary advances.

Table 3: Reinforcement-learning configuration for Plus-scale Qwen-CUA.

C.5 Distributed Training and Environment Rollouts

The Plus-scale configuration uses 512 NVIDIA H200 SXM GPUs across 64 eight-GPU nodes. Following the disaggregated RL architecture of `verl` (verl Project, 2024), we allocate 32 nodes to training and 32 to rollout. Training uses TP $= 2$, EP $= 8$, CP $= 4$, and PP $= 8$; rollout serving uses TP $= 8$, data-parallel replication, SGLang (SGLang Project, 2024), and an optimized attention backend. An asynchronous dispatch layer uses a ratio of 2.35 and caps policy-version skew at four optimizer steps. A full 1,000-update run takes approximately five days, corresponding to about 61,440 H200 GPU-hours.

Environment execution is decoupled from model inference and routed to isolated ECS workers from the pool in Section 3.1. A Plus-scale run keeps up to 2,000 environments active at an average utilization above 75%. For each trajectory, the router restores the task snapshot, streams screenshots and native actions, invokes the evaluator after termination, and resets the worker. Isolation prevents state leakage and allows trajectories from the same task group to execute in parallel.

D Benchmark Evaluation Details

Qwen-CUA and Qwen3.7 use the agent scaffold described in Section 2. We compare them with GPT-5.5 through OpenAI's official computer-use interface (OpenAI, 2026b) and Claude Opus 4.8 through Anthropic's computer-use tool (Anthropic, 2026b). Across all eight benchmarks, we use each benchmark's designated task split and native scoring protocol.

E Qwen for Chrome Showcases

Figure 10 follows a single real-world trajectory from the internal Chrome extension. Given a natural-language request, the deployed agent plans and executes multi-step browser interactions while keeping the user in control at decision points that commit changes or incur cost. The frames follow the editorial transitions in the demonstration video, preserving the original browser interface and visible agent trace.

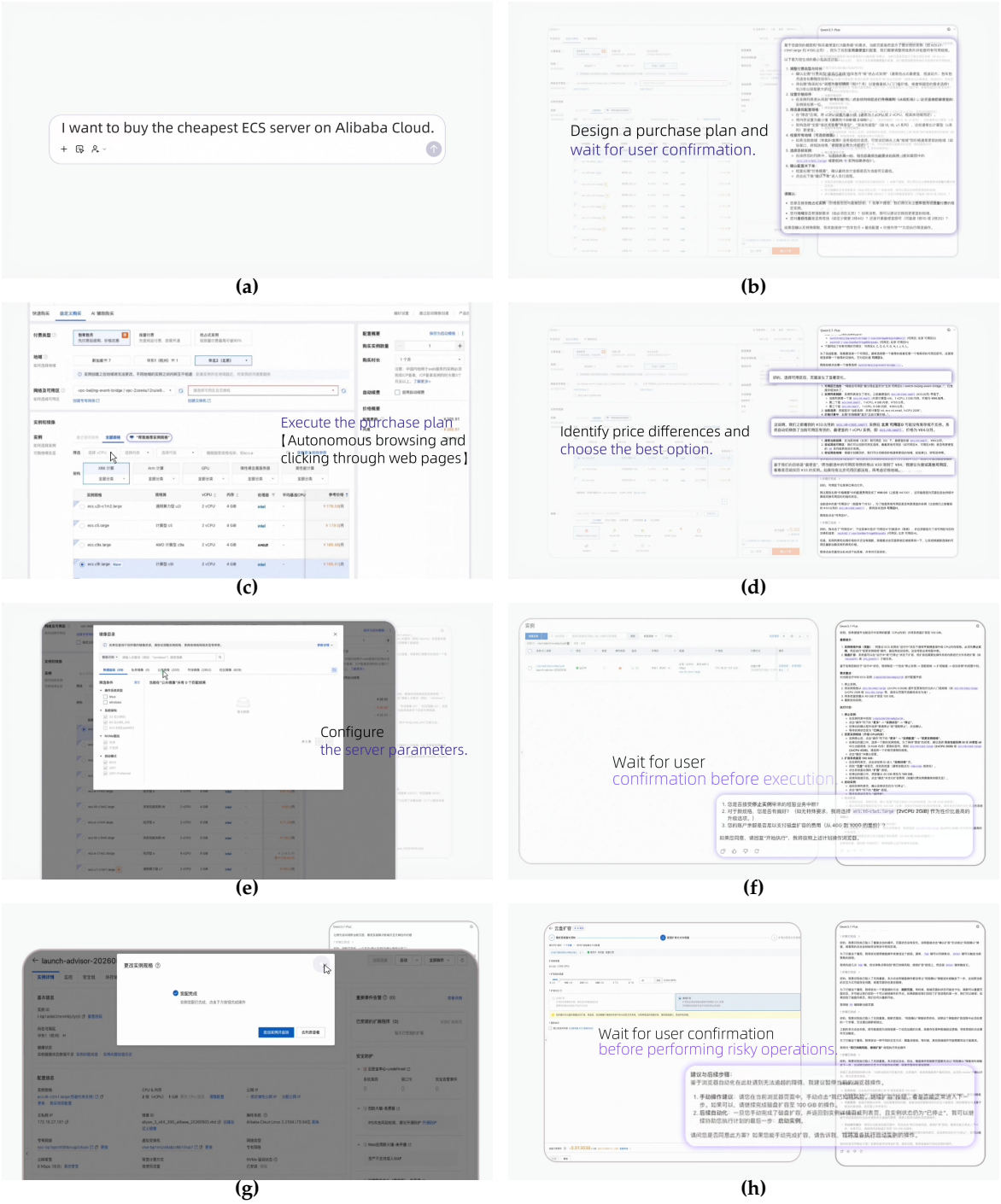


Figure 10: A representative Qwen for Chrome trajectory. (a) A user requests the cheapest ECS server. (b) The agent proposes a purchase plan and waits for confirmation. (c–d) It autonomously browses the site, compares prices, and selects an option. (e) It configures the server. (f) Before applying the configuration, it requests confirmation. (g) The specification change completes successfully. (h) The agent pauses again before a risky disk expansion operation.

F Broader Computer-Use Related Work

Visual grounding and native agent models. The transition from structured web agents to general computer-use agents has required models to understand dense, high-resolution interfaces and ground language directly to screen coordinates. SeeClick, UGround, and OS-ATLAS study this grounding problem across web, desktop, and mobile interfaces (Cheng et al., 2024; Gou et al., 2024; Wu et al., 2024); CogAgent and Ferret-UI develop vision-language models specialized for GUI and screen understanding (Hong et al., 2023; You et al., 2024). More recent native agents integrate grounding with planning, reasoning, and action prediction. Representative systems include AutoGLM, Aguvis, UI-TARS, UI-TARS-2, and OpenCUA (Liu et al., 2024; Xu et al., 2024; Qin et al., 2025; Wang et al., 2025a;b; Bai et al., 2025). Qwen-CUA belongs to this latter family, with a screenshot-only observation interface and native keyboard-and-mouse control.

Training data, tasks, and environments. Computer-use supervision has been collected from human demonstrations, tutorial-guided replay, unlabeled interaction videos, and synthesized grounding examples (Wang et al., 2025b; Xu et al., 2025; Lu et al., 2025; Xie et al., 2025). A complementary line of work constructs executable environments and verifiable tasks for online learning. ZeroGUI and related exploration-based systems use learned evaluators to broaden task coverage, while GUI-GENESIS, InfiniteWeb, and AutoWebWorld synthesize controllable web environments with machine-checkable outcomes (Yang et al., 2025a; Cao et al., 2026; Zhang et al., 2026; Wu et al., 2026b). Gym-Anything broadens application coverage, and CUA-Gym jointly scales reusable environments, task states, and executable reward functions; SCALECUA further combines verifiable task synthesis with capability-aware sampling for online training (Aggarwal et al., 2026; Wang et al., 2026a; Lv et al., 2026).

Rollout infrastructure. Large-scale agent learning also requires environment orchestration to be decoupled from GPU-side optimization. NanoRollout exposes heterogeneous agent harnesses and environment backends through a shared rollout service used by evaluation, trajectory distillation, and reinforcement-learning clients (Wang et al., 2026c). This systems abstraction complements cloud-scale environment pools by allowing rollout workers and trainers to scale independently.

Reinforcement learning and iterative improvement. GUI-R1 explores rule-based reinforcement learning for grounded GUI actions, whereas UI-TARS-2 studies multi-turn reinforcement learning in interactive GUI environments (Xia & Luo, 2025; Wang et al., 2025a). ZeroGUI highlights the importance of reward quality for stable online optimization (Yang et al., 2025a). EvoCUA-1.5 and SCALECUA extend this line with multi-turn online optimization, adaptive data selection, and asynchronous or segmented rollout processing (Huang et al., 2026; Lv et al., 2026). SEAgent and EvoCUA instead emphasize self-evolution: an improved policy returns to the environment, generates new experience, and learns again from selected trajectories (Sun et al., 2025b; Xue et al., 2026). These directions motivate Qwen-CUA's combination of state-based rewards, long-horizon trajectory optimization, and iterative rollout filtering.

Computer-use evaluation. Web benchmarks progressed from offline demonstrations in Mind2Web to interactive, reproducible environments such as WebArena, VisualWebArena, and WorkArena (Deng et al., 2023; Zhou et al., 2023; Koh et al., 2024; Drouin et al., 2024). Desktop evaluation expanded through OmniACT, OSWorld, OSWorld-Verified, Windows Agent Arena, and Computer Agent Arena (Kapoor et al., 2024; Xie et al., 2024; Bonatti et al., 2024; XLANG Lab, 2025; Wang et al., 2026b). Subsequent benchmarks stress business processes, long repetitive execution, personalized state, professional workflows, and long-horizon cross-application work (Wornow et al., 2024; Wu et al., 2026a; Jang et al., 2026; Sun et al., 2025a; Yuan et al., 2026). Our evaluation complements these perspectives with broad capability, efficiency, deployment, and hybrid-tool analyses.

Safety and robustness. As computer-use agents act on consequential state, evaluation must separate task capability from safe execution. RedTeamCUA studies indirect prompt injection in hybrid web–OS environments, OS-Harm covers deliberate misuse, prompt injection, and model misbehavior, and RiOSWorld measures operational risk in multimodal computer use (Liao et al., 2025; Kuntz et al., 2025; Yang et al., 2025b). These benchmarks motivate explicit confirmation before consequential actions and reporting task success jointly with attack success or other risk measures.